Public editions adapted from real Career Agent Pro outputs

Edição pública adaptada de uma saída real do produto. Material educacional gerado por IA e revisado para publicação. Todos os contextos de arquitetura, equipes, incidentes, requisitos e avaliação são hipóteses didáticas de plataformas fictícias. Não descrevem informações internas confirmadas, questões oficiais, contratação ou estratégia de empresas reais. Sem vínculo ou endosso de Prezensa, Globo, WhatsApp ou Meta.

Coding challenge Android: feed e ciclo de vida

Engenharia Android para plataforma fictícia de streaming. Nível didático: sênior. Este documento não descreve uma vaga real.

Visão Geral do Desafio

Este desafio simula um problema real de produção: implementar uma **tela de feed de conteúdos** para um app de streaming em escala nacional.

Formato sugerido: implemente e explique uma fatia vertical de feed. O roteiro de simulação observa correção, clareza de estado, testes e justificativa de escolhas; não reproduz avaliação de empresa real.

> Na simulação, explique decisões e limitações sem sacrificar correção. Código, testes e comunicação são dimensões complementares desta rubrica didática.

Enunciado

Feed de Conteúdos com Gerenciamento de Estado e Ciclo de Vida

Implemente uma tela de feed que exibe cards de conteúdo (filmes, séries, notícias) carregados de uma API remota.

Requisitos Funcionais

- [ ] Ao abrir a tela, carregar os conteúdos da API e exibi-los em lista
- [ ] Suportar **três estados de UI**: `Loading`, `Success` e `Error`
- [ ] Em caso de erro, exibir mensagem com **botão de retry**
- [ ] O estado deve **sobreviver à rotação de tela** sem refazer a chamada de rede
- [ ] Ao navegar e voltar, o **estado cacheado deve ser restaurado** sem nova requisição

Requisitos Técnicos

- [ ] **Kotlin** com Coroutines e Flow
- [ ] **ViewModel** para gerenciamento de estado
- [ ] **StateFlow** para expor o estado à UI
- [ ] Camada de dados **abstraída por interface de repositório**
- [ ] Dependências **injetáveis via construtor** para facilitar testes

---

Estrutura de Arquivos Recomendada

A estrutura abaixo é uma sugestão didática. Outras decomposições são válidas quando os contratos e os testes permanecem claros.

```
├── ui/
│   ├── FeedFragment.kt
│   └── FeedViewModel.kt
├── domain/
│   ├── model/Content.kt
│   └── repository/ContentRepository.kt     ← interface
├── data/
│   └── repository/ContentRepositoryImpl.kt
└── test/
    └── FeedViewModelTest.kt
```

Contratos de Código

Interface do Repositório

```
interface ContentRepository {
    suspend fun getContents(): List<Content>
}

data class Content(
    val id: String,
    val title: String,
    val thumbnailUrl: String,
    val type: ContentType
)

enum class ContentType { MOVIE, SERIES, NEWS }
```

Estados da UI

```
sealed class FeedUiState {
    object Loading : FeedUiState()
    data class Success(val contents: List<Content>) : FeedUiState()
    data class Error(val message: String) : FeedUiState()
}
```

Comportamento esperado do ViewModel

```
// Inicialização  → emite Loading, depois Success ou Error
// retry()        → volta para Loading e refaz a requisição
// StateFlow      → mantém o último estado para novos coletores (ex: após rotação)
```

Edge Cases - Casos de análise e discussão

Edge Case 1 - Rotação de tela durante carregamento

Cenário: O usuário rotaciona a tela enquanto o estado ainda é `Loading`.

Pergunta de simulação:

> *"O que acontece com a Coroutine em execução quando o Fragment é destruído? E quando o novo Fragment é criado após a rotação?"*

Discussão proposta: O `viewModelScope` garante que a Coroutine **não é cancelada** na rotação - porque o ViewModel sobrevive a ela. O novo Fragment se inscreve no `StateFlow` e recebe o estado atual imediatamente, **sem nova requisição de rede**.

Edge Case 2 - App vai para background durante a requisição

Cenário: A requisição está em andamento quando o usuário pressiona Home.

Pergunta de simulação:

> *"O que acontece com o Flow coletado no Fragment? E a Coroutine no ViewModel continua rodando?"*

Discussão proposta: lifecycleScope é cancelado quando seu Lifecycle é destruído; launch simples não interrompe automaticamente coleta em STOPPED. No Fragment, use viewLifecycleOwner.lifecycleScope com viewLifecycleOwner.repeatOnLifecycle(STARTED): o bloco de coleta é cancelado abaixo de STARTED e recriado ao retornar. viewModelScope não é cancelado por rotação, mas termina quando o ViewModel é limpo; não sobrevive à morte do processo.

> No Fragment, use viewLifecycleOwner.lifecycleScope e viewLifecycleOwner.repeatOnLifecycle(STARTED). Isso cancela o bloco abaixo de STARTED e recria a coleta ao retornar; Lifecycle STARTED não garante visibilidade física em todas as situações. Não confundir com lifecycleScope.launch simples.

Edge Case 3 - Erro de rede seguido de retry com sucesso

Cenário: A primeira chamada falha com `IOException`. O usuário clica em retry e a segunda chamada tem sucesso.

Pontos a analisar:

- Transição correta de estados: `Error` → `Loading` → `Success`

- Ausência de race condition em cliques rápidos no botão de retry

Pergunta de simulação:

> *"Como você garante que cliques rápidos em retry não disparam múltiplas Coroutines simultâneas?"*

Edge Case 4 - Lista vazia retornada pela API

Cenário: A API retorna `200 OK` com uma lista vazia.

Pontos a analisar: A distinção entre `Success(emptyList())` e `Error`. Uma lista vazia é um **estado de sucesso** - não uma falha. A UI deve tratar esse caso separadamente (ex: tela "nenhum conteúdo disponível").

Pergunta de simulação:

> *"Você modelaria lista vazia como Success ou adicionaria um quarto estado ao sealed class?"*

> **Atenção:** Não há resposta certa aqui. O facilitador da simulação quer ver se você **tem uma opinião fundamentada** - e sabe justificá-la.

Edge Case 5 - Testabilidade do ViewModel

Cenário: O facilitador da simulação pede que você escreva um teste unitário do `FeedViewModel`.

```
@OptIn(ExperimentalCoroutinesApi::class)
@Test
fun `feed exposes loading and success`() = runTest {
    val dispatcher = StandardTestDispatcher(testScheduler)
    Dispatchers.setMain(dispatcher)
    try {
        val repository = FakeContentRepository(
            response = listOf(Content("1", "Notícia de exemplo", "", ContentType.NEWS))
        )
        // Contrato do exemplo: uiState inicia em Loading e o init
        // agenda a busca em viewModelScope, usando a Main substituída.
        val viewModel = FeedViewModel(repository)
        viewModel.uiState.test {
            assertEquals(FeedUiState.Loading, awaitItem())
            advanceUntilIdle()
            val success = awaitItem() as FeedUiState.Success
            assertEquals(1, success.contents.size)
            cancelAndIgnoreRemainingEvents()
        }
    } finally {
        Dispatchers.resetMain()
    }
}
```

Pontos para autoavaliação:

- Controlar Main e testScheduler com StandardTestDispatcher; observar estado inicial antes de avançar o scheduler. O exemplo depende do contrato de inicialização descrito, pois StateFlow conflaciona estados.

- Dependências injetadas via construtor (nunca hardcoded)
- Familiaridade com a biblioteca **Turbine** para testar sequências de Flow

Critérios didáticos de avaliação

Critérios centrais do exercício

Critério	O que é avaliado
viewModelScope correto	Nunca usar `GlobalScope` ou Coroutines sem escopo controlado
StateFlow exposto como imutável	Expor `StateFlow`, nunca `MutableStateFlow` diretamente
Repositório como interface	Injetado via construtor - nunca instanciado dentro do ViewModel
Ordem correta dos estados	`Loading` deve ser emitido *antes* da chamada de rede, não depois

Alta Relevância

Critério	O que é avaliado
repeatOnLifecycle no Fragment	Coleta lifecycle-safe - não apenas `lifecycleScope.launch`
Retry sem race condition	Serializar requests e impedir publicação de resultados obsoletos; cancelamento local não desfaz efeito remoto
Distinção entre lista vazia e erro	Clareza na modelagem de estados
Fake em vez de Mock no repositório	Mais robusto para testar Flow; indica maturidade em design de testes

Diferenciais - Tópicos de aprofundamento

Critério	O que é avaliado
Turbine para testar Flow	Conhecimento de ferramentas específicas do ecossistema Kotlin
UseCase entre ViewModel e Repository	Separação de camadas explícita, mesmo que simples
Verbalização de trade-offs	Ex: *"Poderia usar SharedFlow para eventos únicos, mas escolhi StateFlow porque o estado precisa ser retido"*
Conexão com o produto real	Relacionar o exercício a comportamentos observáveis em apps de streaming

Exemplo de raciocínio em nível sênior

Um profissional no nível esperado **não começa a escrever código imediatamente**. O raciocínio segue esta progressão:

---

1. Modelagem de estado - antes de qualquer código

> *"Preciso mapear os estados possíveis da tela: Loading, Success e Error. Vou usar sealed class - o compilador garante exhaustividade no `when`. Para lista vazia, tratarei como `Success(emptyList())` e deixarei a UI decidir o que renderizar. Mantém a camada de dados simples e a UI expressiva."*

---

2. StateFlow vs SharedFlow

> StateFlow retém estado para coletores e funciona bem com ViewModel na rotação; não persiste por si só após morte do processo. SharedFlow pode modelar eventos, mas replay, ausência de coletores e confirmação de consumo precisam ser definidos; não garante entrega exatamente uma vez.

---

3. Coleta de Flow no Fragment

> Use viewLifecycleOwner.lifecycleScope e viewLifecycleOwner.repeatOnLifecycle(STARTED). O bloco de coleta é cancelado e reiniciado conforme o ciclo da view; não é uma pausa transparente da mesma execução. Evite acessar uma binding já destruída.

---

4. Retry sem race condition

> Para o retry, serialize as intenções e trate cancelamento cooperativo: cancelar o Job anterior não garante que um efeito remoto cessou. Considere cancelAndJoin, mutex ou identificador de geração para impedir resposta antiga de sobrescrever estado novo; conflate não é substituto para deduplicação de efeitos.

---

5. Testabilidade como resultado do design

> *"Como o repositório é injetado via construtor pela interface, crio um `FakeContentRepository` nos testes sem Mockito. Para testar a sequência de estados emitidos pelo Flow, uso a biblioteca Turbine - feita exatamente para isso."*

> Na simulação, use o tempo para demonstrar modelagem, implementação e limites. Qualidade de comunicação complementa correção técnica; não existe promessa de resultado seletivo.

Limites e evidências da implementação

Separe estado de UI, recursos do player e dados persistidos. ViewModel ajuda em mudanças de configuração, mas não substitui restauração após morte do processo. Serviços também podem ser encerrados; definir recuperação e persistir apenas o necessário. No feed, diferencie cache vazio, dado antigo, atualização em andamento e falha de atualização.

Compare arquiteturas por invariantes e evidências: testes com scheduler controlado, cancelamento, callbacks após destruição da view, alteração de conta, relógio incorreto e falhas parciais. Metas numéricas são parâmetros do exercício; medir em uma matriz sintética de dispositivos e cargas antes de afirmar que uma alternativa cumpre o orçamento.

Referências técnicas e escopo

Referências de APIs públicas para estudo; não são evidência sobre arquitetura ou processos de empresas reais. Os contratos Kotlin são exemplos educacionais, sem implementação de produtos privados.

- Android: coroutines e ciclo de vida
- Android: WorkRequests e constraints
- Media3: ExoPlayer e application thread

Career Agent PRO

Edição pública adaptada de saída real. Profundidade técnica, hipóteses explícitas e decisões verificáveis.

www.career-agent-pro.com

support@main.career-agent-pro.com

© 2026 Career Agent PRO. Todos os direitos reservados.

ADAPTED PUBLIC EDITION